

## Machine Problem 4: Virtual Memory Management and Memory Allocation

Nithin Gowtham Saravanan

UIN: 337001954

CSCE611: Operating Systems

### Assigned Tasks

**Main:** Completed.

**Bonus Option 1:** Testing of VM pool by disabling the flag `_TEST_PAGE_TABLE_`

**Bonus Option 2:** Not applicable.

**Bonus Option 3:** Not applicable.

**Bonus Option 4:** Not applicable.

### System Design

The objective of this machine problem is to extend the existing kernel paging infrastructure (from MP3) with dynamic page allocation and virtual memory pool management. The design is categorized into 3 parts

1. The page table pages (and even page table directories) are allocated in mapped memory, i.e. memory above 4MB in our case. These frames are handled by the process frame pool.
2. Prepare the page table to support virtual memory
3. To implement a simple virtual-memory allocator

### Paging Architecture

The paging system follows a x86-style two-level hierarchy:

- Page Directory (PDE): Points to page tables.
- Page Tables (PTE): Contain mappings of virtual to physical pages.

This project is designed based on Recursive Page Table Lookup (i.e.,) the last entry (1023) of the page directory recursively maps to itself, enabling a self-referential view where both PDEs and PTEs can be accessed via logical addresses without physical translation. This allows quick computation of `PDE_address()` and `PTE_address()`.

### Method of Implementation:

1. **Contiguous Frame Pool (CFP):** Implemented a 2-bit/entry bitmap (Free, Used, Head-of-Sequence) to support contiguous frame allocation and release; maintained a static list of pools for `release_frames()` lookup.

2. **Metadata Placement:** Stored the CFP bitmap inside a designated info frame (or pool's first frame if `_info_frame_no==0`) and marked it HoS/Used; computed info size via `needed_info_frames()` (2 bits/frame).
3. **Physical Layout Setup:** Created two pools in kernel.C: Kernel pool (2 MB at 2 MB) and Process pool (28 MB at 4 MB). Marked the 15–16 MB hole as inaccessible in the process pool.
4. **Paging Init (PageTable::init\_paging):** Registered kernel/process pools and shared region size (4 MB). Deferred enabling until PD/PTs exist.
5. **Page Directory & Recursive Mapping:** Allocated a PD frame (process pool), zeroed it, set `PDE[1023]` to point to the PD itself (recursive trick). Built identity-mapped PT(s) for the 4 MB shared region.
6. **Address Translation Helpers:** Implemented `PDE_address(addr)` and `PTE_address(addr)` using the recursive windows (`0xFFFFF000` and `0xFFC00000`) to get pointers to PDE/PTE in VA space.
7. **Page Fault Handling:** On fault, if VM pools are registered, validate the VA with `VMPool::is_legitimate`. Created a PT on demand if PDE is not present; then allocated a data frame and installed the PTE; flushed TLB for that VA.
8. **Virtual Memory Pools (VMPool):** Added first-fit sub allocator over a `[base, size]` VA range, tracking free/allocated regions (page granularity). `allocate()` returns a VA; first touch faults and maps frames. `release()` frees all pages in the region via `PageTable::free_page`.
9. **Validation Modes (via `_TEST_PAGE_TABLE_`):**
  - **Defined:** Demand-page writes at VA 4 MB; expect PT creation + 2 data pages, readback verifies.
  - **Undefined:** Create Code/Heap pools; `new[]` uses VMPool; touches fault-map pages; `delete[]` releases pages; both pools pass integrity checks.

## Code Description

This implementation modifies the following files:

- `cont_frame_pool.C`
- `page_table.H`
- `page_table.C`
- `vm_pool.H`
- `vm_pool.C`
- `kernel.C`

**cont\_frame\_pool.C**

The same file used for MP2, and MP3 is used with little modifications to improve the implementation of memory manager.

- Updated the constructor to correctly handle both cases where the bitmap frame lies within the pool (`_info_frame_no` inside range) or outside it (`_info_frame_no == 0`). This ensures that only the valid metadata frame is marked HoS and reduces accidental marking of unrelated frames.

```
// Mark metadata/info frame(s) as used
if (_info_frame_no == 0)
{
    set_state(0, FrameState::HoS);
    nFreeFrames--;
}
else if (_info_frame_no >= base_frame_no && _info_frame_no < base_frame_no + nframes)
{
    unsigned long local = _info_frame_no - base_frame_no;
    set_state(local, FrameState::HoS);
    nFreeFrames--;
}
```

- Replaced hard assertions in `get_frames()` with safe return-based error handling for invalid or insufficient allocations, preventing kernel crashes during transient memory fragmentation or low-memory conditions.
- Refined `mark_inaccessible()` to decrement `nFreeFrames` only when a frame transitions from Free to Used/HoS, eliminating double decrements and keeping pool statistics accurate across overlapping metadata and hole markings.

```
FrameState prev = get_state(local);
if (fno == _base_frame_no)
    set_state(local, FrameState::HoS);
else
    set_state(local, FrameState::Used);
if (prev == FrameState::Free)
    nFreeFrames--;
```

**page\_table.H**

- Introduced a pointer “`vm_pools_head`” to point to the head of linked list of registered VMPools
- Function prototypes for recursive lookup helpers **PDE\_address** and **PTE\_address**

```
/* --- Recursive lookup helpers --- */

unsigned long * PDE_address(unsigned long addr); /* address of PDE for addr */
unsigned long * PTE_address(unsigned long addr); /* address of PTE for addr */
```

**page\_table.C**

1. **Recursive Mapping Setup** - Added constants RECURSIVE\_PDE\_BASE and RECURSIVE\_PTE\_BASE to enable direct logical access to all PDEs and PTEs using the recursive mapping trick (PDE[1023] → PD).

→ Simplifies updates and avoids unsafe physical dereferences.

```
+ /* Helper constants for recursive mapping:
+  * PDE logical base: 0xFFFFF000 (maps page directory entries as 1023th PDE -> PD)
+  * PTE logical base: 0xFFC00000 (maps page tables via recursive PDE trick)
+  */
+ static const unsigned long RECURSIVE_PDE_BASE = 0xFFFFF000UL;
+ static const unsigned long RECURSIVE_PTE_BASE = 0xFFC00000UL;
+
```

2. **init\_paging()** - Initializes paging globals (kernel\_mem\_pool, process\_mem\_pool, shared\_size), separating kernel and process pools.  
→ Provides a consistent setup before any PageTable is created or paging enabled.
3. **PDE\_address() / PTE\_address()** - New helper functions returning logical pointers to a page's PDE or PTE via the recursive window.

→ Enables safe and fast manipulation of page-table entries.

```
+ /* Return pointer (logical) to the PDE of 'addr' using recursive trick */
+ unsigned long * PageTable::PDE_address(unsigned long addr)
+ {
+     unsigned int pd_idx = (addr >> 22) & 0x3FF;          // top 10 bits
+     unsigned long addr_pde = RECURSIVE_PDE_BASE + (pd_idx * sizeof(unsigned long));
+     return (unsigned long *)addr_pde;
+ }
+
+ /* Return pointer (logical) to the PTE of 'addr' using recursive trick */
+ unsigned long * PageTable::PTE_address(unsigned long addr)
+ {
+     unsigned int pd_idx = (addr >> 22) & 0x3FF;
+     unsigned int pt_idx = (addr >> 12) & 0x3FF;
+     unsigned long addr_pte = RECURSIVE_PTE_BASE
+                             + ((unsigned long)pd_idx << 12) // select page table page
+                             + (pt_idx * sizeof(unsigned long)); // select entry
+     return (unsigned long *)addr_pte;
+ }
```

4. **PageTable() Constructor** - Now allocates all page-directory and table frames from the process memory pool (not kernel).

Builds identity-mapped tables for the shared 4 MB region and sets up recursive mapping (PDE[1023] = PD).

```
page_directory[1023] = (pd_frame << 12) | P_PRESENT | P_RW;
```

```
69 - unsigned long pt_frame = kernel_mem_pool->get_frames(1); 113 + unsigned long pt_frame = process_mem_pool->get_frames(1);
```

→ Ensures proper memory ownership and enables recursive lookups.

5. **handle\_fault()** - Rewritten to support both test modes:

- Page-table test: demand-pages directly.
- VMPool test validates faults only for legitimate allocated regions.

Dynamically allocates missing PTs and data frames from the process pool, zeroes via recursive mapping, and flushes TLB selectively.

→ Enables clean demand paging and safe region enforcement.

```
/* Legitimacy check only if any pools are registered (VM-pool mode).
   In pure page-table test mode (no pools), allow demand paging. */
if (current_page_table->vm_pools_head != nullptr)
{
    VMPool *vp = current_page_table->vm_pools_head;
    bool legitimate = false;
    while (vp)
    {
        if (vp->is_legitimate(fault_addr))
        {
            legitimate = true; break;
        }
        vp = vp->next;
    }
    if (!legitimate)
    {
        Console::puts("Page Fault: Invalid access (address not allocated by any VMPool). Aborting.\n");
        assert(false);
        return;
    }
}
```

6. **register\_pool()** - Implements a linked list of registered VMPools for address-legitimacy checks in handle\_fault().

→ Allows PageTable to differentiate valid from invalid page faults.

```
void PageTable::register_pool(VMPool * _vm_pool)
{
    Console::puts("PageTable::register_pool called\n");
    assert(_vm_pool != nullptr);
    _vm_pool->next = vm_pools_head;
    vm_pools_head = _vm_pool;
    Console::puts("\tVMPool registered with PageTable\n");
}
```

## 7. free\_page()

Frees a physical frame mapped to a given virtual page and clears its PTE with invlpg.

→ Supports VMPool's release logic and keeps memory consistent.

```
void PageTable::free_page(unsigned long _page_no)
{
    Console::puts("PageTable::free_page called for page ");
    Console::putui(_page_no);
    Console::puts("\n");

    unsigned long virtual_addr = _page_no * PAGE_SIZE;

    /* Get pointer to PTE via recursive mapping */
    unsigned long * pte_ptr = PTE_address(virtual_addr);
    unsigned long pte = *pte_ptr;

    if ((pte & P_PRESENT) == 0) {
        Console::puts("\tPage not present (nothing to free)\n");
        return;
    }

    /* Extract physical frame number from PTE */
    unsigned long phys_frame = (pte >> 12) & 0xFFFFF;

    Console::puts("\tReleasing physical frame ");
    Console::putui(phys_frame);
    Console::puts("\n");

    /* Release the frame back to the frame pool (absolute frame number) */
    ContFramePool::release_frames(phys_frame);

    /* Invalidate the PTE and flush that page from the TLB */
    *pte_ptr = 0;
    invlpg((void *)virtual_addr);

    Console::puts("\tPage freed and TLB entry invalidated\n");
    return;
}
```

## 8. Frame Pool Policy Update

All PD/PT/data allocations now draw from the process frame pool to maintain kernel–process separation and adhere to MP4 specs.

→ Aligns paging with assignment design and ensures isolation.

**vm\_pool.H**

- The `allocated_regions` and `free_regions` arrays (with `Region` structs) provide a simple bookkeeping mechanism to manage memory allocation within each virtual memory pool — tracking which pages are in use and which remain available.
- Each `VMPool` maintains its own `base_address`, `size_bytes`, and `frame_pool`, ensuring isolation between different virtual memory regions (e.g., code, heap) and mapping them independently through their associated `PageTable`.
- Storing regions in pages (`base_page`, `length`) simplifies operations such as allocation, release, and legitimacy checks, aligning directly with paging granularity.
- The next pointer allows `VMPool` objects to be registered as a linked list within `PageTable`, enabling the fault handler to validate whether a faulting address belongs to a legitimate allocated region.

```
class VMPool { /* Virtual Memory Pool */
public:
    /* Next pointer used by PageTable to maintain registered-pools list (MP4) */
    VMPool * next = nullptr;

private:
    /* -- DEFINE YOUR VIRTUAL MEMORY POOL DATA STRUCTURE(s) HERE. */
    unsigned long base_address;    // virtual start (bytes)
    unsigned long size_bytes;      // pool size in bytes
    ContFramePool * frame_pool;    // backing physical frame pool (used by PageTable on faults)
    PageTable * page_table;        // page table for this address space

    /* Region bookkeeping arrays (simple implementation) */
    static const unsigned int MAX_REGIONS = 256; // limited but sufficient
    struct Region {
        unsigned long base_page;    // page number (virtual) relative to entire address space
        unsigned long length;       // in pages
    };

    Region allocated_regions[MAX_REGIONS];
    unsigned int n_allocated;

    Region free_regions[MAX_REGIONS];
    unsigned int n_free;
```



## vm\_pool.C

### VMPool::VMPool()

- This constructor initializes the virtual memory pool by defining its **base virtual address** and **total size in bytes**, linking it to a specific **frame pool** and **page table**.
- It divides the address space into page-sized units, marks the entire pool as initially free, and inserts the pool into the page table's registry (register\_pool) so that page faults can be validated against legitimate address ranges.
- Console messages provide initialization feedback, including base address, size, and total pages.

```
/* Initialize free region to entire pool (in pages, using absolute virtual pages) */
unsigned long base_page = base_address / PageTable::PAGE_SIZE;
unsigned long total_pages = bytes_to_pages(size_bytes);

free_regions[0].base_page = base_page;
free_regions[0].length = total_pages;
n_free = 1;

/* Register ourselves with the PageTable so page-fault handler can query legitimacy */
page_table->register_pool(this);
```

### VMPool::allocate()

- This function implements a first-fit allocation policy to locate the first available free region large enough to satisfy the request.
- It converts the requested byte size to the number of required pages, updates the free and allocated region lists, and returns the starting virtual address of the allocated block.
- If no contiguous free region can satisfy the request, the function logs a failure and returns 0.
- This operation does not immediately map physical frames — pages are mapped lazily upon page faults handled by the PageTable.

```
unsigned long need_pages = bytes_to_pages(_size);

/* First-fit: Find a free region with enough pages */
for (unsigned int i = 0; i < n_free; ++i)
{
    if (free_regions[i].length >= need_pages)
    {
        /* Use the first portion of this free region */
        unsigned long alloc_base_page = free_regions[i].base_page;
        /* Update free region */
        free_regions[i].base_page += need_pages;
        free_regions[i].length -= need_pages;
        if (free_regions[i].length == 0)
        {
            /* Remove this free region by shifting */
            for (unsigned int j = i; j + 1 < n_free; ++j)
                free_regions[j] = free_regions[j+1];
            n_free--;
        }
    }
}
```



```

/* Add to allocated regions */
assert(n_allocated < MAX_REGIONS); // otherwise out of bookkeeping space
allocated_regions[n_allocated].base_page = alloc_base_page;
allocated_regions[n_allocated].length = need_pages;
n_allocated++;

unsigned long vaddr = alloc_base_page * PageTable::PAGE_SIZE;
Console::puts("\tVMPool::allocate returning virtual addr ");
Console::putui(vaddr);
Console::puts("\n");
return vaddr;
}
}

```

### VMPool::release()

- The release() function deallocates a previously allocated memory region identified by its starting address.
- It first locates the corresponding region in the allocated\_regions list, then iterates over all its pages and calls PageTable::free\_page() for each to release the associated physical frames.
- Once all pages are freed, the region metadata is moved back into the free\_regions list (without coalescing adjacent blocks for simplicity).
- This ensures both the page table and virtual memory bookkeeping remain consistent.

```

/* Compute start page and find allocated region */
unsigned long start_page = _start_address / PageTable::PAGE_SIZE;
int idx = -1;
for (unsigned int i = 0; i < n_allocated; ++i)
{
    if (allocated_regions[i].base_page == start_page) {
        idx = (int)i;
        break;
    }
}

if (idx == -1) {
    Console::puts("\tVMPool::release: start address not found in allocated regions\n");
    assert(false);
    return;
}

unsigned long length = allocated_regions[idx].length;

/* For each page in the region, ask page table to free if present */
for (unsigned long p = 0; p < length; ++p)
{
    unsigned long page_no = allocated_regions[idx].base_page + p;
    page_table->free_page(page_no);
}

/* Move this region from allocated array to free array (no coalescing) */
assert(n_free < MAX_REGIONS);
free_regions[n_free].base_page = allocated_regions[idx].base_page;
free_regions[n_free].length = allocated_regions[idx].length;
n_free++;

/* Remove allocated region by shifting */
for (unsigned int j = idx; j + 1 < n_allocated; ++j)
    allocated_regions[j] = allocated_regions[j+1];
n_allocated--;

```

**VMPool::is\_legitimate()**

- Checks whether a given virtual address falls within a currently allocated region of the pool.
- It iterates through all active regions in `allocated_regions` and returns true if the address lies between the region's start and end pages.
- This function is crucial for the page fault handler — it prevents illegal memory access by ensuring only addresses belonging to legitimate allocated regions are serviced.

```
bool VMPool::is_legitimate(unsigned long _address)
{
    unsigned long page = _address / PageTable::PAGE_SIZE;

    for (unsigned int i = 0; i < n_allocated; ++i)
    {
        unsigned long start = allocated_regions[i].base_page;
        unsigned long end = start + allocated_regions[i].length; // exclusive
        if (page >= start && page < end)
            return true;
    }
    return false;

    Console::puts("Checked whether address is part of an allocated region.\n");
}
```

**kernel.C**

The flag “`_TEST_PAGE_TABLE_`” has been enabled and disabled to test the scenarios of just the page table and just the virtual memory pool respectively.

```
/* BY DEFAULT WE TEST THE PAGE TABLE IN MAPPED MEMORY!
   (UNCOMMENT THE FOLLOWING LINE TO TEST THE VM Pools! */
#define _TEST_PAGE_TABLE_
```

```
/* BY DEFAULT WE TEST THE PAGE TABLE IN MAPPED MEMORY!
   (UNCOMMENT THE FOLLOWING LINE TO TEST THE VM Pools! */
//#define _TEST_PAGE_TABLE_
```

## Bonus Option 1:

*Testing of VM pool by disabling the flag `_TEST_PAGE_TABLE_`*

To test the VM pool, disable the flag “`_TEST_PAGE_TABLE_`” and the log report is stored in `kernel_log_vm_pool.txt` by executing the command  
**make run > kernel\_log\_vm\_pool.txt 2>&1**

## TESTING

### Scenario 1: `_TEST_PAGE_TABLE_` is enabled to test the page table

- Shows paging initialization, creation of the kernel and process frame pools, page directory + table allocation, and activation of paging.
- A single page fault sequence occurs: the page fault handler detects a missing table, allocates it, and maps one data frame from the frame pool.
- The run ends with “*Test Passed! Congratulations!*” — confirming that page-table construction, page-fault handling, and frame mapping are correct.

This validates that ContFramePool and PageTable operate correctly without the VM-pool layer: frame allocation, PTE setup, and TLB invalidation all behave exactly as required in the handout.

```
nithingowtham25@Nithin:~/MachineProblems_OS/MachineProblems_git_repo/MP4_Sources$ make run
Page Directory loaded into CR3
This Page Table is now active
Loaded page table

Enabling Paging in CR0
PG bit set, paging is now enabled!

Hello World!
EXCEPTION DISPATCHER: exc_no = <14>
Page Fault triggered
Missing Page Table. Allocating new one...
Executing get_frames
Success: found contiguous block from local frames 2 to 2
New Page Table allocated and linked
Page not present. Allocating data frame...
Executing get_frames
Success: found contiguous block from local frames 3 to 3
Mapped faulted VA using a frame from its VMPool
EXCEPTION DISPATCHER: exc_no = <14>
Page Fault triggered
Page not present. Allocating data frame...
Executing get_frames
Success: found contiguous block from local frames 4 to 4
Mapped faulted VA using a frame from its VMPool
DONE WRITING TO MEMORY. Now testing...
Test Passed! Congratulations!
YOU CAN SAFELY TURN OFF THE MACHINE NOW.
One second has passed
One second has passed
One second has passed
One second has passed
One second has passed
One second has passed
```

**The complete log is available in `kernel_log_page_table.txt`**

**Scenario 2: `_TEST_PAGE_TABLE_` is disabled to test the virtual memory pool**

1. **Setup & Registration** - After paging is enabled and the shared region is identity-mapped, two VMPools are constructed (e.g., code @ 512 MB, heap @ 1 GB). Each pool registers with the PageTable via `register_pool`, enabling the page-fault handler to validate accesses with `is_legitimate`.
2. **Allocation Pattern (First-Fit, Page-Granular)** - The test harness repeatedly calls `VMPool::allocate(size_in_bytes)` with increasing sizes. The pool converts bytes → pages and carves the first sufficiently large free region, returning the **starting virtual address** (VA). No frames are grabbed here—mapping is **lazy**.
3. **Demand Paging on First Touch**  
Writes to the returned VA trigger page faults:
  - If the PDE is missing, the handler allocates a **new page table** (from the process pool), links it, flushes CR3, and zeroes the PT page via the recursive mapping window.
  - For each faulted page, the handler allocates a **data frame** (process pool), writes the PTE (present | RW), and `invlpg` the VA.
  - Logs show the sequence: “Page Fault triggered → Missing PT (once per new 4 MB region) → Page not present → Mapped faulted VA”.
4. **Legitimacy Enforcement** - Before mapping, the handler walks the VMPool list; `is_legitimate(fault_addr)` must return true (address ∈ an allocated region). This prevents accidental/illegal mapping outside pool allocations. In Scenario 2 the check **must** pass for all accesses; otherwise the handler aborts.
5. **Data Integrity Check** - For each allocated block, the test writes deterministic values and reads them back in reverse to validate correctness of mappings across multiple pages and tables. Success here confirms the end-to-end path: VA allocation → fault → PT/PT entry creation → physical frame mapping → usable memory.
6. **Release & Frame Reclamation** - The test then calls `VMPool::release(start_va)`. For every page in the region, `PageTable::free_page` clears the PTE, calls `ContFramePool::release_frames` on the physical frame (HoS + trailing Used frames), and `invlpgs` the VA. Log shows “Freeing HoS/Used... → Page freed and TLB entry invalidated”, confirming frames return to the pool(s).
7. **Allocator Behavior & Limits Observed** - The pool uses **first-fit** and **no coalescing** of free regions (intentionally simple). Despite fragmentation risk, the scripted pattern (allocate → touch → verify → release) repeatedly succeeds, demonstrating correct bookkeeping (allocated\_regions / free\_regions) and stable interaction with the page-table and frame-pool layers.

These steps together validate that, under VM-Pool mode, allocations are legitimate, demand-paged correctly, memory reads/writes are consistent, and resources are fully reclaimed.

```

Page not present. Allocating data frame...
Executing get_frames
Success: found contiguous block from local frames 7 to 7
Mapped faulted VA using a frame from its VMPool
EXCEPTION DISPATCHER: exc_no = <14>
Page Fault triggered
Page not present. Allocating data frame...
Executing get_frames
Success: found contiguous block from local frames 8 to 8
Mapped faulted VA using a frame from its VMPool
VMPool::release called for address <1074315264>
PageTable::free_page called for page <262284>
Releasing physical frame <1028>
ContFramePool::release_frames called for frame 1028
Freeing HoS at local frame 4
Page freed and TLB entry invalidated
PageTable::free_page called for page <262285>
Releasing physical frame <1029>
ContFramePool::release_frames called for frame 1029
Freeing HoS at local frame 5
Page freed and TLB entry invalidated
PageTable::free_page called for page <262286>
Releasing physical frame <1030>
ContFramePool::release_frames called for frame 1030
Freeing HoS at local frame 6
Page freed and TLB entry invalidated
PageTable::free_page called for page <262287>
Releasing physical frame <1031>
ContFramePool::release_frames called for frame 1031
Freeing HoS at local frame 7
Page freed and TLB entry invalidated
PageTable::free_page called for page <262288>
Releasing physical frame <1032>
ContFramePool::release_frames called for frame 1032
Freeing HoS at local frame 8
Page freed and TLB entry invalidated
VMPool::release completed - Released region of memory.
Test Passed! Congratulations!
YOU CAN SAFELY TURN OFF THE MACHINE NOW.
One second has passed
One second has passed

```

Since multiple commands are used for the purpose of debugging, the screenshot of the last few messages is enclosed to ensure the proper working of virtual memory pool.

***The complete log for this test scenario is available in kernel\_log\_vm\_pool.txt***